

A23 基于大语言模型的文档理解与多源数据融合系统

竞赛执行方案

1 项目总体目标与问题理解

在现实办公场景中，大量信息存在于非结构化文档中，例如：

- 合同文件
- 项目报告
- 技术说明
- 客户反馈
- 业务材料

这些文档通常以 Word、Excel、Markdown 或纯文本的形式存在，其内容结构不统一，信息分散且冗余。在传统工作流程中，工作人员需要逐个阅读这些文档，并手动将关键信息整理到 Excel 表格或数据库中。这一过程具有三个明显特点：

第一，重复性强。

在实际工作中，很多表格需要不断从不同文档中提取类似的信息，例如项目名称、金额、负责人等字段。工作人员往往需要多次重复阅读文档并进行人工记录。

第二，效率低下。

人工阅读文档的速度有限，而企业或机构在实际工作中可能需要处理数十甚至上百份文档。如果完全依赖人工整理，整个流程会变得非常缓慢。

第三，容易产生错误。

人工处理文档信息时容易出现遗漏、错录或格式不统一的问题。例如金额单位错误、日期格式混乱等情况。

因此，本项目的目标是构建一个**能够自动理解文档内容并将关键信息转化为结构化数据的智能系统**。该系统需要完成以下核心能力：

首先，系统能够读取多种格式的文档，包括 Word、Excel、Markdown 和 TXT 文件，并将其转换为统一的文本结构。

其次，系统能够从这些文本中自动识别关键信息，例如项目名称、合同金额、日期和单位名称等。

再次，系统能够根据用户提供的模板表格，将这些信息自动填入对应字段，从而生成结构化的业务表格。

最终，系统需要在保证准确率的前提下完成这一过程，并在规定时间内完成任务，以满足竞赛评测指标。

从系统角度来看，该项目并不是一个单一算法问题，而是一个完整的信息处理系统，其核心价值在于：

将非结构化文档自动转化为结构化数据。

2 系统整体架构设计

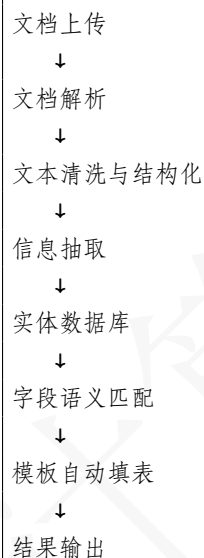
为了实现上述目标，系统需要设计为一个**模块化的信息处理系统**。模块化设计的主要目的有两个：

- 第一，使系统结构清晰，方便团队成员分工开发。
- 第二，使各模块能够独立优化，从而逐步提高系统整体性能。

本系统将整体流程划分为五个核心模块：

1. 文档解析模块
2. 信息抽取模块
3. 数据结构化模块
4. 字段语义匹配模块
5. 自动填表模块

系统整体数据流如下：



下面对每一个模块的功能和设计逻辑进行详细说明。

3 文档解析模块设计

文档解析模块是整个系统的第一步，其作用是将不同格式的文档转换为统一的数据结构。在实际办公环境中，文档格式是高度多样化的。例如：

- Word 文档通常包含段落、标题和表格

- Excel 文件以表格形式组织数据
- Markdown 文档具有标题层级结构
- TXT 文档则是纯文本

如果不进行统一解析，后续模块将无法对这些文档进行统一处理。因此，文档解析模块需要完成以下三个关键任务。

首先，需要识别文档的结构。

例如 Word 文档中通常包含多个段落以及嵌入表格，而 Markdown 文件则具有标题层级结构。系统需要能够识别这些结构信息，并将其记录下来。

其次，需要提取文档的文本内容。

所有后续信息抽取算法都依赖文本内容，因此系统需要将文档中的文字信息完整提取出来，并去除无关的格式符号。

第三，需要生成统一的数据表示格式。

不同类型文档在解析后都需要转化为统一的 JSON 数据结构，以便后续模块使用。

例如，一个文档在解析后可以表示为：

```
{
  "doc_id": "001",
  "paragraphs": [
    "项目名称为智慧校园建设项目",
    "合同金额为50万元"
  ],
  "tables": [
    [ ["字段", "内容"], ["项目名称", "智慧校园建设项目"] ]
  ],
  "raw_text": "...
}
```

通过这种方式，后续模块无需关心文档原始格式，而只需要处理统一的结构化文本。

从团队协作角度来看，这一模块是整个系统的基础模块。如果文档解析不稳定，那么后续所有模块都无法正常运行。因此，这一模块需要优先开发并反复测试。

4 信息抽取模块设计

在文档解析完成之后，系统需要从文本中识别出关键信息。这一过程称为信息抽取。

信息抽取的核心目标是将自然语言中的内容转化为明确的实体字段。例如，在以下文本中：

“本合同项目名称为智慧校园建设项目，合同金额为 50 万元。”

系统需要识别出两个实体：

- 项目名称：智慧校园建设项目
- 合同金额：50 万元

为了实现这一目标，本系统采用三层抽取机制。

第一层是规则抽取。

规则抽取主要通过正则表达式实现，用于识别具有明显格式特征的信息。例如金额、日期和编号等字段通常具有固定格式，通过规则可以快速识别。

第二层是实体识别模型。

对于一些无法通过简单规则识别的信息，例如公司名称或项目名称，可以使用命名实体识别模型进行识别。

第三层是大语言模型补全。

在某些复杂情况下，文本表达方式可能比较灵活，此时可以利用大语言模型进行语义理解，从而识别隐藏的信息。

通过这三层机制结合，可以显著提高信息抽取的准确率。

抽取结果需要记录来源文档以及置信度，例如：

```
{
  "entity": "合同金额",
  "value": "500000",
  "source_doc": "doc1",
  "confidence": 0.91
}
```

这些信息将被存入系统数据库，以供后续字段匹配模块使用。

5 字段语义匹配模块设计

在实际应用中，模板表格中的字段名称与文档中的字段表达往往并不一致。例如，模板表格可能使用“合同金额”这一字段，而文档中可能写作“合同总金额”或“项目预算”。

如果系统仅依赖字符串匹配，就无法正确识别这些字段。因此，本系统需要引入语义匹配机制。

语义匹配的核心思想是将文本转化为向量表示，然后通过向量相似度来判断两个字段是否表达相同含义。

具体流程如下：

首先，对模板字段生成语义向量。

例如“合同金额”这一字段会被转换为一个向量表示。

其次，对抽取到的实体字段生成向量。

例如“合同总金额”同样会被转换为向量。

最后，通过计算两个向量之间的相似度，判断它们是否匹配。如果相似度超过设定阈值，则认为该实体可以填入该字段。

为了进一步提高准确率，还需要结合规则进行校验。例如金额字段必须包含数字，日期字段必须符合日期格式。

通过语义匹配与规则校验结合，可以显著降低字段匹配错误的概率。

6 数据结构化与知识池设计

在信息抽取模块完成之后，系统已经可以从文档中识别出若干实体信息，例如项目名称、合同金额、日期、单位名称等。但是，这些信息如果只是零散地保存在内存中，是无法支撑后续复杂查询与字段匹配的。因此，系统需要设计一个统一的数据结构，并建立一个可以长期保存数据的“知识池”。

所谓知识池，本质上是一个用于存储文档信息的数据集合，它记录了每个实体信息的来源文档、文本位置以及置信度等信息。通过这种方式，系统能够实现以下几个功能：

第一，可以快速查询实体信息。

例如，当模板表格需要填写“合同金额”这一字段时，系统可以在知识池中搜索所有与金额相关的实体，并找到最可能的候选值。

第二，可以避免重复解析文档。

在比赛流程中，系统需要先上传一批文档，然后再多次上传模板表格。如果每一次填写表格都重新解析所有文档，系统的运行时间将会非常长。通过知识池机制，可以做到“文档只解析一次”，之后的查询全部在数据库中完成。

第三，可以记录信息来源，提高系统可信度。

在展示系统时，如果系统能够指出某个字段来自哪个文档的哪个段落，那么评委会更容易相信系统结果的可靠性。

为了实现这些目标，本系统采用统一的 JSON 数据结构表示实体信息。例如，一个实体记录可以表示为：

```
{
  "entity_type": "合同金额",
  "value": "500000",
  "source_document": "contract1.docx",
  "paragraph_index": 12,
  "confidence": 0.91
}
```

所有抽取到的实体都会存入数据库中。数据库可以使用 SQLite 或 MongoDB 实现，其主要存储内容包括：

- 文档 ID
- 文档解析文本
- 抽取实体
- 实体置信度
- 实体来源位置

通过这一结构，系统可以在后续模块中快速完成字段匹配与数据查询。

从系统设计角度来看，知识池模块承担着“数据中枢”的作用。所有抽取结果都会存入该模块，而后续字段匹配和自动填表模块则从该模块读取数据。因此，该模块需要具有良好的查询性能和清晰的数据结构。

7 自动填表模块设计

在完成文档解析、信息抽取以及字段匹配之后，系统已经获得了所有需要填入模板表格的数据。接下来需要完成的任务是将这些数据写入模板文件中。

自动填表模块的核心目标是根据用户上传的模板表格，自动识别其中的字段名称，并将对应数据填入表格中。

在实际应用中，模板表格通常是 Excel 或 Word 文件。例如，一个 Excel 表格可能包含如下字段：

项目名称	合同金额	签署日期	负责人
------	------	------	-----

系统需要首先读取模板表格中的字段名称，然后利用字段语义匹配模块找到对应的数据，并将其写入表格中。

在技术实现方面，可以使用 Python 的 openpyxl 库来操作 Excel 文件。具体流程如下：

首先，系统读取 Excel 文件，并获取第一行的字段名称。

其次，系统根据字段名称在知识池中查询对应的实体数据。

最后，将匹配到的数据写入 Excel 表格对应单元格。

在写入数据时，还需要进行格式处理。例如：

- 金额统一为数字格式
- 日期统一为 YYYY-MM-DD 格式
- 文本内容去除多余空格

此外，系统还需要处理一些特殊情况。例如，如果某个字段没有找到匹配数据，系统可以填入“未找到数据”或保持空值。这样可以避免系统运行时发生错误。

自动填表模块是整个系统的最终输出模块，因此需要重点测试其稳定性。在比赛现场，评委会将上传多个模板文件进行测试，如果系统能够稳定生成正确的表格结果，将显著提升整体评分。

8 系统架构与性能优化

在前面的模块设计中，我们主要关注的是系统功能。然而，在竞赛评测中，除了准确率之外，系统响应时间也是一个重要指标。因此，在系统设计过程中还需要考虑性能优化问题。

首先，需要避免重复计算。

在比赛流程中，系统通常会先上传一批文档，然后再多次上传模板表格。如果每次填写表格都重新解析文档和抽取信息，那么系统的运行时间将会大幅增加。因此，系统需要设计缓存机制，将解析后的文档信息存入数据库中，从而实现“一次解析，多次使用”。

其次，需要优化语义匹配速度。

在字段匹配过程中，系统需要计算多个字段之间的相似度。如果每次都重新计算 embedding 向量，将会浪费大量时间。因此，可以提前计算实体向量，并存入数据库中，从而在匹配时只需要进行向量相似度计算。

第三，可以采用多线程处理。

在处理大量文档时，可以使用多线程技术同时解析多个文件，从而减少整体处理时间。例如，当系统同时上传十个文档时，可以启动多个线程并行解析，从而提高效率。

通过这些优化措施，可以显著缩短系统运行时间，从而满足竞赛要求的时间限制。

9 五人团队协作与依赖关系

为了保证项目能够顺利完成，团队成员必须按照模块划分进行协作。不同成员之间既需要明确分工，又需要保持良好的沟通。

负责人 1 主要负责文档解析模块。该模块是整个系统的基础，其他模块都依赖于其输出。因此，该模块需要优先完成，并保证稳定性。

负责人 2 负责信息抽取模块，其工作依赖于负责人 1 提供的解析文本。在负责人 1 完成基础解析功能之后，负责人 2 即可开始开发抽取算法。

负责人 3 负责字段语义匹配模块，其任务是根据模板字段在知识池中寻找对应实体。该模块需要依赖负责人 2 的抽取结果。

负责人 4 负责系统架构整合与性能优化。其主要任务是将前面三个模块整合为完整系统，并通过缓存机制、多线程技术等方式优化系统性能。

负责人 5 负责系统前端界面以及竞赛展示材料。该成员需要根据后端接口设计用户界面，并制作项目演示视频和 PPT。

从开发顺序来看，系统模块之间存在如下依赖关系：

文档解析 → 信息抽取 → 字段匹配 → 系统整合 → 前端展示

如果某个模块没有完成，将会影响后续模块的开发。因此，团队需要定期进行代码整合与测试，确保各模块能够正常协作。

10 项目开发时间规划

为了确保项目能够按时完成，团队需要制定明确的开发计划。整个项目周期可以划分为三个阶段。

在第一阶段，团队需要完成系统基础架构搭建。这一阶段的主要任务包括文档解析模块开发以及基础信息抽取功能实现。目标是让系统能够完成基本的文档读取和简单信息识别。

在第二阶段，团队需要重点优化系统功能。这一阶段需要引入语义匹配机制，并改进信息抽取算法，从而提高系统准确率。

在第三阶段，团队需要重点进行系统测试与性能优化。在这一阶段，需要模拟比赛流程进行测试，并确保系统在不同类型文档下都能稳定运行。

通过这样的阶段划分，可以确保项目开发过程有条不紊地推进。

11 测试与评估方案

为了确保系统能够满足竞赛要求，团队需要提前设计测试方案。测试过程需要尽量模拟比赛环境。

首先，需要准备一批测试文档，包括 Word、Excel、Markdown 和 TXT 文件。

其次，需要设计若干模板表格，用于测试系统自动填表能力。

然后，将所有文档一次性上传系统，并测试系统填写多个模板表格的表现。

测试指标主要包括两个方面：

第一是准确率。

准确率表示系统填入表格的数据是否正确。

第二是响应时间。

响应时间表示系统从接收模板到生成结果所需时间。

通过多次测试，可以不断优化系统算法和性能。

12 项目风险与应对策略

在项目开发过程中，可能会遇到一些风险。例如，信息抽取算法可能无法识别某些特殊表达方式，导致字段提取错误。针对这一问题，可以增加规则库，并结合大模型进行语义理解。

另一种风险是系统响应时间过长。如果系统在处理大量文档时运行缓慢，可以通过缓存机制和多线程技术进行优化。

此外，在比赛现场可能出现未知文档格式，因此系统需要具备一定的鲁棒性，例如在解析失败时给出提示，而不是直接崩溃。

13 竞赛答辩准备

在竞赛答辩阶段，评委通常关注三个问题：

第一，系统是否真正解决了实际问题。

第二，系统技术方案是否合理。

第三，项目是否具有创新性。

因此，在准备答辩材料时，需要重点展示系统架构、核心算法以及实际应用场景。通过清晰的演示流程，可以让评委快速理解系统价值。

14 团队成员职责说明（详细版）

为了确保系统开发过程高效有序，团队成员需要按照模块进行明确分工。每位成员不仅需要负责自身模块的开发，还需要与其他成员保持持续沟通，以保证系统整体架构的一致性。

14.1 负责人 1：文档解析模块负责人

负责人 1 承担整个系统最基础的任务，即实现多种文档格式的统一解析。由于系统需要处理 Word、Excel、Markdown 和 TXT 等不同类型的文件，而这些文件的内部结构差异较大，因此必须通过专门的解析模块将其转换为统一的数据结构。

在实际开发中，负责人 1 首先需要研究不同文档格式的读取方式。例如，Word 文档可以通过 python-docx 库进行解析，而 Excel 文件则可以通过 pandas 或 openpyxl 进行读取。Markdown 文件可以按照标题层级进行结构解析，而 TXT 文件则需要通过文本分段方式进行处理。

解析过程中不仅需要提取文本内容，还需要识别文档中的结构信息。例如，Word 文档中的标题可以反映内容层级，而表格中往往包含重要信息。如果这些结构信息被忽略，将会影响后续的信息抽取模块。

因此，在设计解析模块时，需要同时输出三类信息：

第一类是文档段落内容，即连续的文本段落。

第二类是表格结构信息，即文档中所有表格的内容。

第三类是文档整体文本，用于后续语义分析。

这些信息最终需要统一保存为 JSON 数据结构，以便后续模块直接调用。

此外，负责人 1 还需要负责文本清洗工作。例如，一些文档中可能包含多余空格、特殊符号或编码错误，如果不进行处理，将会影响信息抽取算法的准确性。因此，在解析阶段就需要完成基础清洗工作。

从项目整体角度来看，文档解析模块是所有模块的基础。如果这一模块不稳定，后续模块将无法正常工作。因此，该负责人需要在开发初期优先完成解析模块，并通过大量测试确保系统能够稳定解析各种文档格式。

14.2 负责人 2：信息抽取模块负责人

负责人 2 的任务是从解析后的文本中识别关键信息，并将其转换为结构化实体数据。这一过程被称为信息抽取，是整个系统的核心功能之一。

在设计信息抽取模块时，需要考虑文本表达的多样性。例如，在不同文档中，“合同金额”这一信息可能呈现为“合同金额”、“总金额”或“项目预算”。如果系统只依赖简单的关键词匹配，就可能无法识别这些不同表达方式。

因此，本系统采用多层抽取机制。

首先是规则抽取层。

规则抽取主要用于识别具有明显格式特征的信息。例如金额通常包含数字和单位，而日期通常符合特定格式。通过正则表达式，可以快速识别这些信息。

其次是实体识别层。

对于项目名称、单位名称等信息，可以使用命名实体识别模型进行识别。这样可以提高系统对复杂文本的理解能力。

最后是大语言模型辅助层。

当规则和实体识别无法确定信息时，可以利用大语言模型进行语义理解，从而判断文本中隐藏的实体信息。

在抽取过程中，系统还需要为每个实体记录置信度。置信度反映系统对该信息正确性的判断。如果一个字段存在多个候选值，可以通过置信度进行排序，从而选择最可能的结果。

抽取完成后，所有实体信息需要存入系统数据库，以供后续字段匹配模块使用。

负责人 2 需要与负责人 1 保持密切沟通，因为信息抽取算法的效果往往取决于文档解析质量。如果解析结果出现错误，抽取模块也会受到影响。

14.3 负责人 3：字段语义匹配负责人

负责人 3 负责解决模板字段与文档字段表达方式不一致的问题。这一问题在实际办公场景中非常常见，例如模板中使用“合同金额”这一字段，而文档中可能使用“项目预算”或“总金额”。

如果系统只依赖字符串匹配，就无法识别这些不同表达方式。因此，需要引入语义匹配技术。

语义匹配的基本思想是将文本转换为向量表示，然后通过向量相似度来判断两个字段是否表达相同含义。

在具体实现中，负责人 3 需要使用文本向量模型生成字段 embedding。例如，可以使用 sentence-transformers 模型将字段名称转换为向量。

随后，系统需要计算模板字段向量与实体字段向量之间的相似度。如果相似度超过一定阈值，就认为两个字段具有相同语义。

为了提高匹配准确率，还需要结合规则校验。例如，如果模板字段是金额类型，那么匹配到的实体也必须包含数字信息。

通过这种方式，可以实现模板字段与文档信息之间的自动映射。

负责人 3 的模块依赖负责人 2 提供的实体数据，因此需要在抽取模块完成基础功能后开始开发。

14.4 负责人 4：系统架构与性能优化负责人

负责人 4 负责系统整体架构设计以及性能优化工作。虽然前面几个模块分别实现不同功能，但只有通过合理的系统架构，才能将这些模块整合为一个完整系统。

首先，负责人 4 需要设计系统后端接口，使各模块之间能够通过统一接口进行通信。例如，可以使用 FastAPI 构建后端服务，使文档上传、信息抽取和自动填表等功能能够通过 API 调用实现。

其次，需要设计数据缓存机制。在比赛流程中，文档只需要解析一次，而模板表格可能会多次上传。如果每次都重新解析文档，将会浪费大量时间。因此，可以将解析后的数据存入数据库，并在后续操作中直接调用。

此外，还需要进行性能优化。例如，在处理多个文档时，可以采用多线程方式并行解析，从而减少整体处理时间。

负责人 4 还需要负责系统日志管理与错误处理。如果系统在运行过程中遇到异常情况，需要记录错误信息并给出提示，而不是直接崩溃。

这一模块在项目后期尤为重要，因为只有通过系统整合和性能优化，才能确保系统在比赛环境中稳定运行。

14.5 负责人 5：前端展示与答辩材料负责人

负责人 5 的主要任务是设计系统界面，并负责项目展示材料的制作。虽然前端开发在技术上相对简单，但在竞赛评审中却具有重要作用。

首先，系统需要提供文件上传界面，使用户能够方便地上传文档和模板表格。界面设计应尽量简洁，使评委能够快速理解系统功能。

其次，需要设计信息可视化界面。例如，当系统抽取出某个字段时，可以在界面上标注该字段来自哪个文档的哪个段落。这种可视化方式可以增强系统可信度。

此外，负责人 5 还需要负责制作项目演示视频和 PPT。在竞赛答辩阶段，清晰的展示材料能够帮助评委快速理解项目价值。

因此，负责人 5 需要在项目后期与其他成员密切合作，确保系统展示效果最佳。